Plus longue sous-chaîne répétée (naïf) + recherche KMP

Répétition naïve | Recherche de motif de Knuth-Morris-Pratt

longest_repeated_naive.cpp | kmp_search.cpp | vérifié par exécution, `g++ -O2 -Wall`

1. Objectif

Étant donnés un texte et un motif, résoudre deux problèmes liés :

1. **Plus longue sous-chaîne répétée** : trouver dans UN seul texte `s` le plus long bloc qui apparaît au moins deux fois (par exemple une phrase répétée dans un livre). Les chevauchements sont autorisés.
2. **Recherche de motif (KMP)** : trouver toutes les occurrences d'un motif `pat` dans un texte `txt`, chevauchements inclus, en temps linéaire.

Résultats de référence (les deux programmes ont été exécutés localement) :

- `"banana"` → `"ana"` (len 3, en 1 et 3)
- `"the cat sat. the dog ran. the cat sat."` → `"the cat sat."` (len 12, en 0 et 26)
- `"ABABDABACDABABCABAB"` recherche `"ABABCABAB"` → 1 occurrence à l'indice 10
- `"aaaaa"` recherche `"aaa"` → occurrences en 0, 1, 2 (chevauchements inclus)

2. Plus longue sous-chaîne répétée (méthode naïve)

Signature

```
RepeatResult longestRepeatedNaive(
    const string &s)
```

Retourne le plus long bloc apparaissant au moins deux fois dans `s`, avec sa longueur et ses deux positions de départ. Texte vide → résultat vide.

Idée en 3 points

1. Essayer chaque paire de départ `(i, j)` avec `j > i`.
2. Étendre `k` tant que `s[i+k] == s[j+k]`.
3. Garder le meilleur quadruplet vu jusqu'ici.

Aucun prétraitement, aucune mémoire supplémentaire. Chevauchements autorisés (`"aaaa"` → `"aaa"`).

Fonction testée telle quelle depuis `longest_repeated_naive.cpp:19`.

```
RepeatResult longestRepeatedNaive(const string &s) {
    RepeatResult best;
    for (size_t i = 0; i < s.size(); i++) {
        for (size_t j = i + 1; j < s.size(); j++) {
            size_t k = 0;
            while (j + k < s.size() && s[i + k] == s[j + k])
                k++;
            if (k > best.len) {
                best.len = k;
                best.posA = i;
                best.posB = j;
                best.value = s.substr(i, k);
            }
        }
```

```
    }
    return best;
}
```

Exemple détaillé : `s = "banana"`. Paires de départ sélectionnées :

(i, j)	s[i] / s[j]	extension	len
(0, 1)	b / a	pas de correspondance (`b` ne se répète jamais)	0
(1, 3)	a / a	`ana`, meilleur	3
(2, 4)	n / n	`na`	2
(3, 5)	a / a	`a`	1

Lecture : la paire (0, 1) échoue car `b` ne se répète jamais ; la paire (1, 3) construit `"ana"` (len 3) et devient la meilleure. Les paires suivantes (`"na"`, `"a"`) ne la battent pas, donc la réponse est `"ana"` (en 1 et 3).

Exemple grandeur nature : `"the cat sat. the dog ran. the cat sat."` → `"the cat sat."` (len 12, en 0 et 26) : la phrase entière répétée est détectée telle quelle.

Complexité : double boucle sur les paires `(i, j)` avec `j > i`, chaque extension jusqu'à `L` → temps $O(n^2 \cdot L)$, mémoire supplémentaire `O(1)`. Sur texte aléatoire l'extension casse vite ; le pire cas (`"a"` répété) allonge chaque extension — mais le nombre de paires domine déjà (section 4, tableau A).

Sorties de démonstration vérifiées (`longest_repeated_naive.cpp:37`):

Entrée (`s`)	Sortie	Statut
`banana`	`"ana"` (len 3, en 1 et 3)	OK
`abcdabc`	`"abc"` (len 3, en 0 et 4)	OK
`aaaa`	`"aaa"` (len 3, en 0 et 1)	OK
`abcdef`	`""` (len 0)	OK
`""`	`""` (len 0)	OK
`the cat sat. the dog ran. the cat sat.`	`"the cat sat."` (len 12, en 0 et 26)	OK

3. Recherche de motif de Knuth-Morris-Pratt

Signatures

```
vector<int> computeLPS(const string &pat)
vector<int> kmpSearch(
    const string &txt, const string &pat)
```

`computeLPS` construit la table des préfixes ; `kmpSearch` retourne chaque indice de départ, chevauchements inclus.

Idée clé

En cas de différence, ne pas reculer le pointeur du texte. Replier avec `j = lps[j-1]` et reprendre la comparaison.

L'indice du texte `i` ne recule jamais, donc le travail total reste linéaire.

Fonction de préfixe telle quelle depuis `kmp_search.cpp:10`.

```
vector<int> computeLPS(const string &pat) {
    vector<int> lps(pat.size(), 0);
    int len = 0; // length of current longest prefix-suffix
    for (size_t i = 1; i < pat.size();) {
        if (pat[i] == pat[len]) {
```

```
            lps[i++] = ++len;
        } else if (len > 0) {
            len = lps[len - 1]; // fall back, do not advance i
        } else {
            lps[i++] = 0;
        }
    }
    return lps;
}
```

Recherche telle quelle depuis `kmp_search.cpp:27`.

```
vector<int> kmpSearch(const string &txt, const string &pat) {
    vector<int> occ;
    if (pat.empty())
        return occ;
    vector<int> lps = computeLPS(pat);
    size_t i = 0; // index in txt
    size_t j = 0; // index in pat
    while (i < txt.size()) {
        if (txt[i] == pat[j]) {
            i++;
            j++;
            if (j == pat.size()) {
                occ.push_back((int)(i - j));
                j = lps[j - 1]; // allow overlapping matches
            }
        } else if (j > 0) {
            j = lps[j - 1]; // skip re-comparisons, key KMP idea
        } else {
            i++;
        }
    }
    return occ;
}
```

Table LPS pour `pat = "ABABCABAB"` (sortie mesurée `[0,0,1,2,0,1,2,3,4]`) :

i	pat[i]	lps[i]	sens
0	A	0	un seul caractère, pas de bordure propre
1	B	0	`"AB"` : pas de bordure
2	A	1	`"ABA"` : bordure `"A"`
3	B	2	`"ABAB"` : bordure `"AB"`
4	C	0	rupture, repli à 0
5	A	1	bordure `"A"` à nouveau
6	B	2	bordure `"AB"`
7	A	3	bordure `"ABA"`
8	B	4	bordure `"ABAB"`

Trace de recherche (événements clés) : `txt = "ABABDABACDABABCABAB"`, `pat = "ABABCABAB"`, occurrence en 10.

étape	pos. texte	pos. motif	événement
1	`i = 0..3`	`j = 0..3`	`ABAB` correspond, puis `txt[4] = D` vs `pat[4] = C` : repli `j = lps[3] = 2`
2	`i = 5..9`	`j = 2..`	le deuxième essai échoue vers `txt[8] = C` ; KMP décale via LPS au lieu de reculer `i`

étape	pos. texte	pos. motif	événement
3	`i = 10..18`	`j = 0..8`	correspondance complète de longueur 9 → occurrence en 10
4	fin	—	1 occurrence en {10} ; les motifs à chevauchement repartiraient de `j = lps[8] = 4`

Vérification du chevauchement : `txt = "aaaaa"`, `pat = "aaa"` → occurrences en 0, 1, 2, car après chaque occurrence `j` repart à `lps[2] = 2` au lieu de 0.

Complexité : LPS coûte temps `O(m)` et espace `O(m)` ; la recherche coûte temps `O(n)`. Total temps `O(n + m)`, espace `O(m)`. L'indice du texte avance au plus `n` fois et l'indice du motif recule au plus `n` fois, donc les comparaisons sont bornées par `2n`.

4. Mesures de performance

Machine : `g++ (Ubuntu 15.2.0-16ubuntu1) 15.2.0`, drapeau `-O2`, CPU Intel Core i7-8665U à 1,90 GHz. Chaque mesure est répétée 3 fois et le meilleur temps est gardé (livre entier : 1 seul passage). Le programme n'affiche pas de compteur de comparaisons : seuls le temps, la longueur et la répétition trouvée sont reportés.

Mesures en bref : le temps montre l'échelle ; la mémoire est `O(1)` supplémentaire pour la répétition naïve contre `O(m)` pour la table LPS ; le nombre de paires rend la méthode naïve quadratique (tableau A) tandis que la recherche KMP reste bornée par `2n` comparaisons.

Tableau A — répétition naïve sur Alice au pays des merveilles (`alice_story.txt`, texte du Projet Gutenberg, préfixes de N caractères) :

N	temps (meilleur de 3)	répétition trouvée
2000	6,7 ms	len 21 en 15 et 483 : `"Down the Rabbit-Hole"`
4000	26,9 ms	len 21, idem
8000	108 ms	len 21, idem
16000	442 ms	len 136 en 9173 et 12116 : ornement `"* * *"`
32000	1079 ms	len 136, idem
162949 (livre entier, 1 passage)	27,4 s	len 145 en 118654 et 119452 : refrain `"Will you, won't you..."`

Lecture : le temps est multiplié par 4 environ quand `N` double (6,7 → 26,9 → 108 → 442 ms, rapports 4,0 / 4,0 / 4,1) : le nombre de paires domine et l'extension casse vite sur texte courant. La dernière marche (442 → 1079 ms, rapport 2,4) a capté un moment creux sur une machine partagée ; la tendance reste clairement quadratique. Le livre entier (162949 caractères) prend 27,4 s et révèle le refrain de la chanson (len 145) : le $O(n^2)$ naïf est infaisable en pratique courante — cela motive les méthodes linéaires (tableau des suffixes, automate), hors sujet ici.

Tableau B — recherche KMP contre recherche naïve :

cas	naïve	KMP
adverse, `pat = "aaab"`, `txt = "a" * 50000 + "b"`	199992 cmp / 0.38 ms	99998 cmp / 0.18 ms
classique, occurrence en {10}	29 cmp	23 cmp
échelle n = 20k	49k cmp / 0.12 ms	26k cmp / 0.10 ms
échelle n = 100k	247k cmp / 0.43 ms	132k cmp / 0.31 ms

Lecture : sur l'entrée adverse, le balayage naïf revérifie chaque décalage (199992 contre 99998 comparaisons, 0.38 contre 0.18 ms) car il recule après chaque quasi-correspondance tandis que KMP ne recule jamais. À grande échelle, KMP utilise environ 1,9x moins de comparaisons (49k contre 26k ; 247k contre 132k) et les deux passent à l'échelle linéairement, ce qui confirme le comportement en `O(n + m)`.

5. Conclusion et sources

La répétition naïve est courte, évidemment correcte et adaptée aux petits textes, mais son temps en $O(n^2 \cdot L)$ devient impraticable sur grand texte (1079 ms pour `N = 32000`, 27,4 s sur le livre entier de 162949 caractères). KMP paie une mémoire en `O(m)` pour la table LPS et obtient une recherche linéaire avec au plus `2n` comparaisons, divisant environ par deux le travail du balayage naïf sur les entrées adverses et volumineuses. Règle pratique : naïf pour la clarté et les minuscules entrées, KMP dès que le texte est grand ou que le motif se chevauche lui-même.

Sources :

1. Nayuki — Recherche de motifs de Knuth-Morris-Pratt. https://www.nayuki.io/page/knuth-morris-pratt-string-matching
2. GeeksforGeeks — plus longue sous-chaîne commune (article sur la programmation dynamique).
3. Wikipédia — Problème de la plus longue sous-chaîne commune.